

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 8

Student Name: M.Purna Chandra

Roll No.: A24126552095

Date: 16-08-2026

1. TITLE

Implementing Middleware in Express.js: Request-Response Lifecycle and Request Logging

2. AIM

To understand the architecture and operation of middleware functions in Express.js, and to design and develop a custom logging middleware that intercepts incoming HTTP requests, logs their HTTP method and URL, and passes control through the request-response cycle.

3. OBJECTIVE

The objectives of this experiment are:

- To understand what middleware is in Express.js and its access to the request (req), response (res), and next middleware function (next).
 - To analyze the flow of the request-response lifecycle when middleware is applied.
 - To construct custom application-level logging middleware using `app.use()`.
 - To use the `next()` function to pass control to subsequent route handlers.
 - To test middleware interception across multiple routes (Home '/' and About '/about').
-

4. THEORY

What is Middleware in Express.js?

Middleware functions in Express.js are functions that execute during the lifecycle of an incoming HTTP request before the final route handler sends a response. A middleware function has access to three core arguments:

1. req (Request Object): Represents the incoming HTTP request (headers, query parameters, body, URL, HTTP method).

2. **res (Response Object):** Represents the outgoing HTTP response used to set headers, cookies, status codes, and send data back to the client.
3. **next (Next Function):** A callback function that, when invoked, passes control to the next middleware function in the execution pipeline. If `next()` is not called, the request hangs unless a response is sent.

Request-Response Cycle

When a client sends an HTTP request to the Express server:

1. **Interception:** The request first enters the middleware pipeline in the exact order the middleware functions were registered with `app.use()` or route definitions.
2. **Execution:** The middleware executes custom logic (e.g., logging request details, authentication, header validation, parsing JSON body).
3. **Decision Point:** The middleware can either:
 - Terminate the cycle by sending a response directly to the client (e.g., `res.status(401).send("Unauthorized")`).
 - Pass control forward by calling `next()`, allowing the next middleware or matching route handler to run.
4. **Route Handler & Response:** The matching route handler (e.g., `app.get("/", ...)`) executes and terminates the cycle by sending `res.send(...)` back to the client.

Common Types of Middleware

Application-level Middleware — Bound to an instance of the `app` object using `app.use()` or `app.METHOD()`.

Router-level Middleware — Bound to an instance of `express.Router()`.

Error-handling Middleware — Special middleware defined with four arguments: (`err`, `req`, `res`, `next`).

Built-in Middleware — Bundled directly with Express, such as `express.json()` and `express.static()`.

Third-party Middleware — Installed via NPM, such as `morgan` (logging) and `cors` (cross-origin resource sharing).

5. ALGORITHM / PROCEDURE

- Create a JavaScript file named `middlewareServer.js`.
- Import the `express` module: `const express = require("express")`.
- Initialize the `express` application: `const app = express()`.
- Define the logging middleware using `app.use((req, res, next) => { ... })`.
- Inside the middleware, log the HTTP request method using `req.method`.
- Log the requested path using `req.url`.
- Call `next()` to pass the request to subsequent route handlers.
- Define a route handler for the home page (`app.get("/", ...)`) to send "Welcome to the Home Page".
- Define a route handler for the about page (`app.get("/about", ...)`) to send "This is the About Page".

- Start the server on port 3000 using `app.listen(3000, ...)`.
 - Run the server using `node middlewareServer.js`.
 - Send HTTP requests to `/` and `/about`, observing the server console logs for each intercepted request.
-

6. PROGRAM

middlewareServer.js

```
// =====  
// Express.js Request Logging Middleware Demonstration  
// =====  
  
// Import Express  
const express = require("express");  
  
// Create an Express application  
const app = express();  
  
// -----  
// Middleware for Logging  
// -----  
  
app.use((req, res, next) => {  
  
    // Display the HTTP method and requested URL  
    console.log("Method:", req.method);  
    console.log("URL:", req.url);  
  
    // Move the request to the next middleware/route  
    next();  
});  
  
// -----  
// Home Route  
// -----  
  
app.get("/", (req, res) => {  
  
    // Send response to the client  
    res.send("Welcome to the Home Page");  
});
```

```

});

// -----
// About Route
// -----

app.get("/about", (req, res) => {

    // Send response to the client
    res.send("This is the About Page");
});

// -----
// Start the Server
// -----

app.listen(3000, () => {

    // Display message when server starts
    console.log("Server running at http://localhost:3000");
});

```

7. CODE EXPLANATION

Code / Component	Explanation
<code>app.use((req, res, next) => { ... })</code>	Registers an application-level middleware function that executes for all incoming requests.
<code>req.method</code>	Property containing the HTTP method of the incoming request (e.g., 'GET', 'POST').
<code>req.url</code>	Property containing the path and URL requested by the client (e.g., '/', '/about').
<code>next()</code>	Invokes the next middleware or route handler in the stack; essential to avoid request timeout.
<code>app.get("/", ...)</code>	Route handler executed after middleware passes control when path is '/'.
<code>app.get("/about", ...)</code>	Route handler executed after middleware passes control when path is '/about'.
<code>app.listen(3000, ...)</code>	Binds the server to port 3000 and begins listening

for network connections.

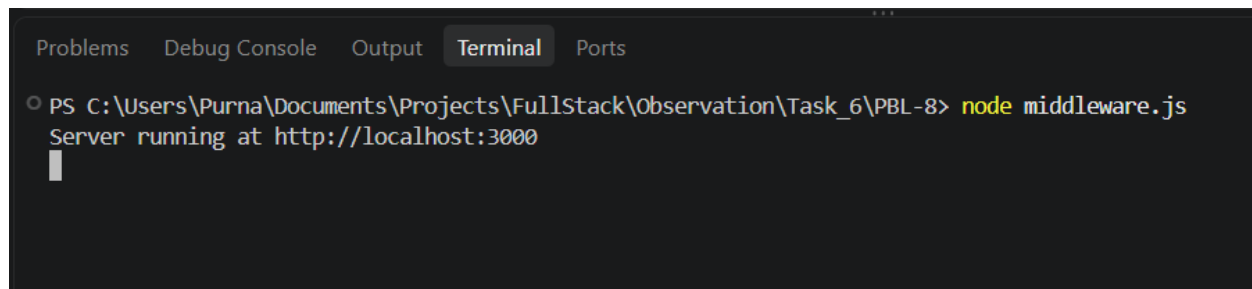
8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Start server with node middlewareServer.js	Server starts and logs "Server running at http://localhost:3000"	Server running on port 3000	Pass
TC-02	Client visits http://localhost:3000/	Console logs Method: GET, URL: /; Client receives "Welcome to the Home Page"	Logged and response received	Pass
TC-03	Client visits http://localhost:3000/about	Console logs Method: GET, URL: /about; Client receives "This is the About Page"	Logged and response received	Pass
TC-04	Client visits undefined route /contact	Console logs Method: GET, URL: /contact; Server returns 404	Middleware logged request prior to 404	Pass

9. OUTPUT

Output 1: Server Startup & Logging Interception in Terminal

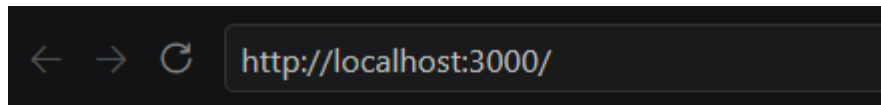
Terminal output showing the server initialization and real-time logs generated by the middleware as requests arrive.



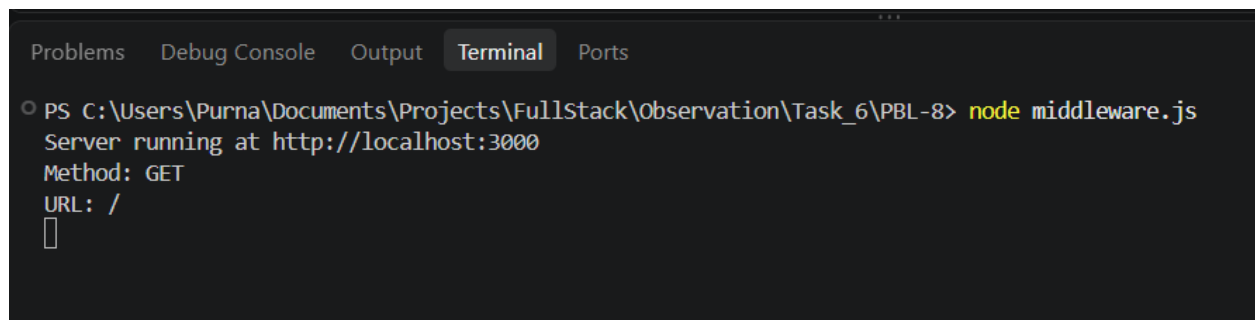
```
Problems  Debug Console  Output  Terminal  Ports
PS C:\Users\Purna\Documents\Projects\FullStack\Observation\Task_6\PBL-8> node middleware.js
Server running at http://localhost:3000
```

Output 2: Client Browser Response and console log and network response for Home route

Text responses received by the client browser for respective endpoints.



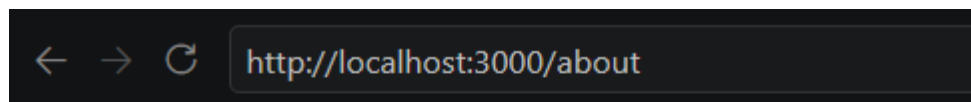
Welcome to the Home Page



Name	Status	Type	Initiator	Size	Time
 localhost	304	docume...	Other	0.2 kB	4.82 s

Output 3: Client Browser Response and console log for About Page

Text responses received by the client browser for respective endpoints.



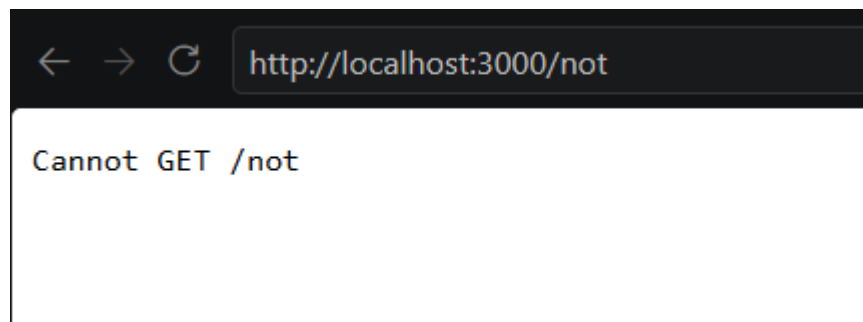
This is the About Page

```
Problems  Debug Console  Output  Terminal  Ports

PS C:\Users\Purna\Documents\Projects\FullStack\Observation\Task_6\PBL-8> node middleware.js
Server running at http://localhost:3000
Method: GET
URL: /
Method: GET
URL: /about
█
```

Name	Status	Type	Initiator	Size	Time
 about	200	docume...	Other	0.3 kB	4 ms

Output 4: Client Browser Response and console log for undefined route



```
Problems  Debug Console  Output  Terminal  Ports
○ PS C:\Users\Purna\Documents\Projects\FullStack\Observation\Task_6\PBL-8> node middleware.js
Server running at http://localhost:3000
Method: GET
URL: /
Method: GET
URL: /about
Method: GET
URL: /not
[]
```

Name	Status	Type	Initiator	Size	Time
✖ not	404	docume...	Other	0.4 kB	6 ms

10. RESULT

Thus, the concept of middleware in Express.js and its role in the request-response cycle was comprehensively explained and implemented. A custom logging middleware application was successfully developed to intercept incoming HTTP requests, log their methods and URLs, and seamlessly pass control to subsequent route handlers using `next()`. All test cases passed with expected outputs.